

CLP Linear Programming Solver

Parameter Tuning Guide

COIN-OR CLP / CBC

Based on source-code analysis of CLP

and the empirical study by Vilas Boas et al. (ITOR 2019)

Sources: `ClpParameters.cpp`, `ClpModel.cpp`, `ClpSimplex{Dual,Primal}.cpp`,
`ClpInterior.cpp`, `ClpSolve.cpp`, `CbcOrClpParam.cpp`, `Cbc_C_Interface.cpp`,
`ClpSimplexOther.cpp` (the `guess()` function), and `Osi/OsiFeatures.cpp`.

The ★ symbol marks high-impact parameters.
Default values shown in **green monospace**.

Contents

1	Overview of CLP's LP Solvers	2
1.1	How the default algorithm is selected	2
2	Parameters Common to All Solvers	2
2.1	★ Scaling — <code>-scaling</code>	2
2.2	★ Presolve — <code>-presolve</code> and <code>-passPresolve</code>	3
2.3	★ Substitution limit — <code>-substitution</code>	3
2.4	★ Refactorisation frequency — <code>-maxFactor</code>	3
2.5	Tolerances — <code>-dualTolerance</code> , <code>-primalTolerance</code>	4
2.6	Sparse factorisation — <code>-sparseFactor</code>	4
2.7	Dual reformulation — <code>-dualize</code>	4
3	Dual Simplex Parameters	4
3.1	★ Dual pivot rule — <code>-dualPivot</code>	4
3.2	★ Positive Edge parameter — <code>-psi</code>	5
3.3	★ Dual bound — <code>-dualBound</code>	6
3.4	★ Perturbation — <code>-perturbation</code> and <code>-pertValue</code>	6
3.5	Dual crash — <code>-crash</code>	7
4	Primal Simplex Parameters	7
4.1	★ Idiot crash — <code>-idiot</code>	7
4.2	★ Sprint crash — <code>-sprint</code>	7
4.3	Primal pivot rule — <code>-primalPivot</code>	8
4.4	Primal weight — <code>-primalWeight</code> / <code>infeasibilityCost</code>	8
5	Barrier (Interior Point) Method Parameters	8
5.1	★ Cholesky factorisation — <code>-cholesky</code>	8
5.2	Regularisation — <code>-gamma</code> and <code>-delta</code>	8
5.3	Crossover — <code>-crossover</code>	9
5.4	Barrier iteration limit — <code>-maxIterations</code>	9
5.5	Barrier perturbation — <code>-pertValue</code> (barrier context)	9
6	Summary Tables	9
6.1	All parameters at a glance	9
7	Empirical Study: Decision Trees for Algorithm Selection	10
7.1	Most discriminating problem features	10
7.2	The depth-3 decision tree	11
7.3	Configurations per leaf and parameters evaluated in Table 2	11
8	Practical Tuning Recommendations	13
8.1	Quick decision guide	13
8.2	Recommended parameter grid for instance-based tuning	13
8.3	Notes for CBC branch-and-bound context	14
9	Default Values Reference	14

1 Overview of CLP's LP Solvers

CLP provides three LP algorithms, selectable via the **-duals**, **-primals**, and **-barrier** switches (or via LPM_Dual/LPM_Primal/LPM_Barrier in the C API).

Method	Default use	Strengths	Weaknesses
Dual Simplex	Root LP + every B&B node	Warm-start from parent basis; fast for LP relaxations	Can be slow on degenerate problems
Primal Simplex	Alternative; crash heuristics available	Better on structured / homogeneous problems; good with idiot crash	No warm-start from dual basis
Barrier (IPM)	Large or difficult root LPs	Scales well; not affected by degeneracy	Requires crossover; no warm-start; expensive Cholesky

1.1 How the default algorithm is selected

When **-solve** is invoked via the C API with LPM_Auto, CLP calls `ClpSimplexOther::guess()` [`ClpSimplexOther.cpp:11037`], a simple heuristic that looks only at *objective coefficient statistics*:

```
sort objective coefficients; compute median(obj) and average(obj)
allInt = all bounded columns are flagged as integer?

if allInt:
    if average <= 0.0086207: -> Primal + idiot=30 + pertv=-1483
    else: -> Primal + idiot=60
else:
    if median <= 0.75: -> Dual + PEsteepest + psi=1.0 + pertv=52
    else: -> Primal + idiot=80
```

The empirical study (Section 7) shows that the most important discriminating feature is actually the *constraint matrix coefficient range* (`aRatioLSA`), which `guess()` does not consider.

2 Parameters Common to All Solvers

2.1 ★ Scaling — -scaling

Default: `automatic`

Options: `off`, `equilibrium`, `geometric`, `automatic`, `dynamic`, `rowsonly`

Source: [`ClpParameters.cpp`, `ClpModel.cpp`]

Scaling transforms the problem so that row and column magnitudes are more uniform, reducing numerical errors and often the iteration count.

- **geometric** — scales by $\sqrt{\max \times \min}$ per row/column. Often best for ill-conditioned matrices. Confirmed important by the paper.

- **equilibrium** — scales by row/column maximum. Cheaper to compute.
- **automatic** — CLP picks the method that minimises element ratio.
- **dynamic** — rescales at every factorisation. Expensive but may help extreme cases.
- **off** — best for numerically clean problems; eliminates unscaling overhead.
- **rows** (or **rowsonly**) — scales rows only; tested in the paper for dual simplex on structured instances.

Recommendation: leave **automatic** for general use. Try **-scaling geo** for ill-conditioned matrices (**aRatioLSA** > 65). The paper found **rows** scaling optimal for a cluster of degenerate dual problems.

2.2 ★ Presolve — **-presolve** and **-passPresolve**

Default: **on** (5 passes)
Options: **on**, **more** (10 passes), **off**
passPresolve range: $[-200, 100]$, default 5
Source: [ClpParameters.cpp:1356]

Presolve removes redundant constraints, fixes variables, and tightens bounds. The **-passPresolve** integer parameter overrides the default number of passes; negative values from the paper (e.g. -81) reduce the pass count below the default, which can speed up B&B nodes where presolve gains are modest.

Recommendation: use **more** or **-passPresolve 20** for large, structured root LPs. Use **-passPresolve -33** or **off** for fast B&B sub-problems. The paper found values of $\{-81, -67, -33, 0, 36, 67\}$ optimal for different instance classes.

2.3 ★ Substitution limit — **-substitution**

Default: **3**
Range: $[0, 10000]$
Source: [ClpParameters.cpp:1400]

Controls the maximum column length for presolve substitution. A value of k means presolve will substitute out any column with at most k non-zeros. Higher values produce more aggressive presolve (fewer rows/columns after presolve, but more fill-in in the remaining matrix).

Recommendation: the paper found **subs=4354** (extremely aggressive) optimal for a class of dual-simplex instances, at the cost of dense post-presolve matrix. Typical useful range: $\{3, 50, 138, 294\}$.

2.4 ★ Refactorisation frequency — **-maxFactor**

Default: **200** (auto-tuned)
Auto-formula: $75 + \lfloor \text{rows}/50 \rfloor$, capped at 1000
Source: [ClpSimplex.cpp:11315 (defaultFactorizationFrequency())]

The LU basis is refactorised every **maxFactor** pivots. Low values increase numerical stability at the cost of more (expensive) factorisations. High values reduce factorisation overhead but accumulate round-off. CLP auto-tunes this by problem size when left at the default of 200.

Recommendation: leave at 200 (triggers auto-tune). For numerically difficult problems try 50–100. For well-conditioned large problems try 400–1000.

2.5 Tolerances — `-dualTolerance`, `-primalTolerance`

Defaults: `1e-7` (parameter file) / `1e-6` (runtime)

Source: `[ClpModel.cpp:132-133]`

dualTolerance: a variable is considered dual-feasible if its reduced cost satisfies $|rc| \leq \varepsilon_d$.

primalTolerance: a constraint is considered satisfied if its violation is below ε_p .

Tightening tolerances below 10^{-8} can cause numerical cycling on ill-conditioned problems. If solutions appear slightly infeasible, *loosen* to 10^{-6} rather than tighten.

2.6 Sparse factorisation — `-sparseFactor`

Default: `on`

Options: `on`, `off`

Source: `[ClpParameters.cpp:1520]`

When `off`, the LU factorisation is treated as dense. Normally this is slower, but the paper found `spars=off` optimal for one cluster of instances where aggressive presolve substitution (`subs=4354`) had already densified the matrix.

2.7 Dual reformulation — `-dualize`

Default: `0` (off)

Options: `0=off`, `1=always`, `2=if rows>cols`, `4=always`

Source: `[ClpParameters.cpp:1322]`

Transposes the problem and solves the LP dual directly. Useful when the problem has many more constraints than variables (tall matrix), so the dual has far fewer rows. The paper evaluated `dualize=1` as one of the configurations for dual simplex instances.

3 Dual Simplex Parameters

The dual simplex is CBC's default LP solver. It solves the LP by maintaining dual feasibility and driving primal infeasibilities to zero.

3.1 ★ Dual pivot rule — `-dualPivot`

Default: `automatic` (= `ClpDualRowSteepest(mode=3)`)

Options: `automatic`, `dantzig`, `partial`, `steep`, `PEsteepest`, `PEdantzig`

Source: `[ClpDualRowSteepest.hpp, CbcSolver.cpp:2751]`

The pivot rule selects which leaving variable (row) enters the basis at each iteration.

Option	Mode	Description
automatic	3	Partial pricing, adaptive to full steepest edge (default constructor <code>ClpDualRowSteepest(3)</code>)
dantzig	—	Largest reduced cost; simple but slow on large problems
partial	2	Partial steepest edge, uninitialized weights
steep	1	Full steepest edge (best anti-degeneracy, most computation)
PEsteepest	3+PE	Positive Edge steepest edge with compatible-variable prioritisation (see Section 3.2)
PEdantzig	—+PE	Positive Edge Dantzig

The **steep** (full steepest edge) maintains exact dual weights and is the strongest anti-degeneracy rule. **PEsteepest** adds the Positive Edge criterion on top.

```
# Use PEsteepest with moderate PE aggressiveness
clp problem.lp -dualPivot PEsteepest -psi 0.5 -duals
```

3.2 ★ Positive Edge parameter — `-psi`

CLI default: `-0.5` (PE inactive in auto mode)

Constructor default (PEsteepest): `0.5`

Range: `(-1.1, 1.1)`

Source: `[ClpPEDualRowSteepest.hpp, CbcOrClpParam.cpp:3591]`

The Positive Edge (PE) criterion [2] prioritises *compatible* variables (those likely to make progress towards optimality) at each pivot. The parameter ψ controls the aggressiveness:

- $\psi \geq 1.0$ — PE completely disabled; reverts to standard steepest edge. The compatible-variable check is skipped entirely `[ClpPEDualRowSteepest.cpp:357]`.
- $0 < \psi < 1.0$ — PE active: a compatible row is preferred if $\text{largestComp} \geq \psi \cdot \text{largestIncompat}$. Smaller ψ = more aggressive (always prefer compatible).
- $\psi < 0$ — PE active; selection criterion becomes $\text{largestComp} \geq |\psi| \cdot \text{largestIncompat}$. The sign has no algorithmic effect once **PEsteepest** is selected.

Two activation paths:

1. `-dualPivot PEsteepest` alone $\Rightarrow$ PE active with $\psi = \text{fabs}(\text{CLI psi}) = 0.5$.
2. `-psi 0.1` (positive, without explicit **PEsteepest**) $\Rightarrow$ PE active for *both* primal and dual with $\psi = 0.1$.

Recommendation: `-dualPivot PEsteepest -psi 0.1` for highly degenerate problems. The paper found `psi=1.0` (PE disabled) as the optimal configuration for some clusters — suggesting that plain steepest edge can be better when the coefficient range is large.

C-API bug in `Cbc_C_Interface.cpp:1897`: the auto-tuner (`guess()`) writes the `psi` value to `int_param[DBL_PARAM_PSI]` instead of `dbl_param[DBL_PARAM_PSI]`. As a result the intended `psi=1.0` is never applied; the default `dbl_param[DBL_PARAM_PSI] = -1.0` is used instead, producing maximally aggressive PE (any compatible row always wins).

3.3 ★ Dual bound — `-dualBound`

Runtime default: 10^{10} (set in `ClpModel` constructor)
Parameter default: 0 (maps to 10^{10} at solve time)
Range: $[10^{-20}, 10^{20}]$
Source: `[ClpModel.cpp:61]`

Bounds the magnitude of dual variables. Any dual variable exceeding this bound is treated as if at its bound, effectively converting unbounded dual variables to finite ones. This prevents the dual from becoming ill-conditioned but can cause the LP to appear infeasible if set too tight.

Recommendation: reduce to 10^6 – 10^8 for ill-conditioned problems to improve numerical stability. Values around 10^9 balance stability and accuracy. Leave at 10^{10} for well-conditioned problems.

3.4 ★ Perturbation — `-perturbation` and `-pertValue`

Default (`perturbation_`): 100 (auto perturbation on)
`pertValue` range: $[-5000, 102]$, default 50
Source: `[ClpModel.cpp:113, ClpSimplexDual.cpp:6379]`

Perturbation adds small random noise to cost coefficients to break degeneracy. The two parameters interact:

`-perturbation` (coarse control)

- 50 — no perturbation (but CLP may still perturb automatically if it detects degenerate objectives).
- 100 — automatic perturbation (default). CLP decides when to perturb.
- 101 / 102 — already perturbed / stop further perturbation.

`-pertValue` (fine-grained control, $[-5000, 102]$) Values 51–61 select the maximum perturbation fraction from a lookup table ($m[\text{pertValue} - 51]$):

<code>pertValue</code>	<code>maxFraction</code>	Effect
50	auto	Standard auto-perturbation
51	10^{-10}	Minimal perturbation
52	10^{-9}	Very conservative (used with <code>PEsteepest</code> by <code>guess()</code>)
56	10^{-5}	Moderate
61	1.0	Large perturbation

Negative values activate a more complex encoding: values ≤ -900 also perturb row costs; values with tens-digit ≥ 1 activate uniform perturbation. For example, `pertValue=-1483` (used by `guess()` for all-integer primal) encodes: row-cost perturbation + uniform change + `maxFraction` = 10^{-5} .

Recommendation: use `-pertValue 52` with `PEsteepest` (conservative, lets PE handle degeneracy). Try `-pertValue 56` or `61` for strongly degenerate primal problems. For the paper’s leaf L5 configuration, `pertValue=81` was optimal.

3.5 Dual crash — `-crash`

Default: `off`

Options: `off, on, solow, lots, idiot1-idiot7`

Source: `[ClpParameters.cpp]`

An initial basis construction heuristic. For dual simplex, the paper found `crash=idiot6` (6 passes of the idiot heuristic as warm-start) optimal for a specific class of degenerate instances. This is unusual since idiot is normally a primal preprocessing technique.

4 Primal Simplex Parameters

The primal simplex maintains primal feasibility and drives dual infeasibilities to zero. Best when a good primal-feasible starting point is available or after idiot crash.

4.1 ★ Idiot crash — `-idiot`

Default: `-1` (auto; use only if problem is suitable)

Range: `[-1, ∞)`; `0 = off`, `n > 0 = n` passes

Source: `[ClpParameters.cpp:1322]`

The idiot heuristic runs a conjugate-gradient-like method (similar to MINOS's crash) that produces a near-primal-feasible starting point before the simplex. It is most effective on *homogeneous* problems with unit elements and structured RHS (e.g. network-flow, binary LPs).

Value	When optimal (from paper)
7	Well-conditioned, near-uniform objective, no integer structure
30	All-integer problems, very small average objective
60	All-integer problems, moderate objective
80	Continuous LPs with median objective > 0.75
100	Ill-conditioned problems with large objective range

Recommendation: `-idiot 60` is a strong default for primal simplex on structured MIP LPs. Try values from `{7, 30, 60, 80, 100}`.

4.2 ★ Sprint crash — `-sprint`

Default: `-1` (auto; off on most problems)

Range: `[-1, ∞)`; `0 = off`, `n > 0 = n` columns per sprint

Source: `[ClpParameters.cpp:1390]`

Sprint (also called *sifting* in CPLEX) is designed for *long and thin* problems — many more columns than rows. CLP solves a sequence of restricted LPs each using a subset of `n` columns, cycling through the full column set.

Recommendation: try `-sprint 1217` or `-sprint 3384` on problems with cols $\gg$ rows (e.g. cutting stock, multi-commodity flow). Values `{1217, 1557, 1804, 3384, 4826}` were explored in the paper.

4.3 Primal pivot rule — `-primalPivot`

Default: `automatic` (= `ClpPrimalColumnSteepest(mode=3)`)

Options: `automatic`, `exact`, `dantzig`, `partial`, `change`, `sprint`, `PEsteepest`

Source: [`ClpPrimalColumnSteepest.hpp`, `CbcSolver.cpp:2784`]

Option	Mode	Description
<code>automatic</code>	3	Adaptive devex (partial → full)
<code>exact</code>	0	Exact devex weights
<code>dantzig</code>	—	Largest reduced cost
<code>partial</code>	4	Partial pricing, dantzig/devex → adaptive
<code>change</code>	1	Full steepest edge
<code>sprint</code>	10	Sprint-based column selection
<code>PEsteepest</code>	+PE	Positive Edge steepest edge

4.4 Primal weight — `-primalWeight` / `infeasibilityCost`

Default: `1010` (internal; parameter default `1010`)

Source: [`ClpParameters.cpp`, `ClpSimplexPrimal.cpp`]

The primal simplex uses a big- M penalty (`infeasibilityCost`) to handle infeasibility. This value controls the relative importance of reducing infeasibility vs. improving the objective. Increasing it makes CLP more aggressive in reaching primal feasibility.

5 Barrier (Interior Point) Method Parameters

The barrier method uses a primal-dual interior point algorithm. It does not maintain a basis and typically requires a crossover step to recover an optimal simplex basis.

5.1 ★ Cholesky factorisation — `-cholesky`

Default: `native`

Options: `native`, `dense`, `UniversityOfFlorida`, `Mumps`, `Wssmp`

Source: [`ClpSolve.cpp:3140-3330` (`cholesky` case switch)]

The Cholesky factorisation is the dominant cost at each barrier iteration. `UniversityOfFlorida` uses the AMD (Approximate Minimum Degree) ordering from the SuiteSparse library, which dramatically reduces fill-in for sparse problems.

If compiled with AMD/SuiteSparse: always use `-cholesky UniversityOfFlorida`. It was the only non-native option in the paper's barrier experiments, confirming its importance. Note: with CHOLMOD available, CLP forces `supernodal=0` because barrier matrices may not be positive definite in early iterations.

5.2 Regularisation — `-gamma` and `-delta`

Defaults: $\gamma = 0$, $\delta = 0$

Source: [`ClpInterior.cpp:61-62`, `ClpSolve.cpp:3290`]

γ and δ are regularisation parameters for the barrier normal equations. They prevent the KKT matrix from becoming singular near the boundary of the feasible region (ill-conditioned /

near-degenerate problems). They are set via **-gamma** and implicitly via the **aggressiveGamma** bitfield in `ClpSolve`:

Level	γ	δ	When to use
off (default)	0	0	Well-conditioned problems
1	10^{-5}	10^{-5}	Mild ill-conditioning
2	10^{-7}	0	Near-degenerate primal
3	0	10^{-5}	Near-degenerate dual
4	10^{-3}	10^{-3}	Strong regularisation

5.3 Crossover — **-crossover**

Default: `on`

Options: `on, off, maybe`

Source: `[ClpSolve.cpp]`

After barrier convergence, crossover moves from the interior point solution to a vertex of the optimal face (a basic feasible solution). This is required for:

- Warm-starting subsequent dual simplex iterations (e.g. B&B nodes).
- Sensitivity analysis (requires a basis).

Use `off` if only the optimal objective value and primal solution are needed (not a basis), saving potentially significant computation.

5.4 Barrier iteration limit — **-maxIterations**

Default: `200` (raised to `1000` if `rows+cols > 10000`)

Source: `[ClpInterior.cpp:103]`

5.5 Barrier perturbation — **-pertValue (barrier context)**

The paper tested barrier-specific **pertValue** values: $\{-208, -61, 50, 51, 56, 61, 102, 208\}$. The same encoding as the dual simplex applies. Values 51–56 are conservative; 102 stops further perturbation; negative values encode complex perturbation patterns including row-cost modification.

6 Summary Tables

6.1 All parameters at a glance

Parameter	Default	Solver	Tuning suggestion
scaling	<code>automatic</code>	All	<code>geo</code> (ill-cond.), <code>rows</code> (structured dual)
presolve	<code>on</code>	All	<code>more</code> for root LP
passPresolve	<code>5</code>	All	$\{-81, -33, 20, 80\}$
substitution	<code>3</code>	All	$\{50, 138, 4354\}$
maxFactor	<code>200 (auto)</code>	All	$\{50-200, 400-1000\}$
dualTolerance	<code>1e-6</code>	All	loosen to <code>1e-5</code> if unstable
primalTolerance	<code>1e-6</code>	All	loosen to <code>1e-5</code> if unstable
sparseFactor	<code>on</code>	All	<code>off</code> after aggressive <code>subs</code>

(continues...)

Parameter	Default	Solver	Tuning suggestion
dualize	0	Dual/Primal	2 (rows $\gg$ cols)
dualPivot	automatic	Dual	PEsteepest (★)
psi	-0.5	Dual/Primal	{0.1, 0.5} (PE active); 1.0 (PE off)
dualBound	1e10	Dual	{ 10^6 , 10^8 , 10^9 } (ill-cond.)
perturbation	100	Dual	50 (off), 100 (auto)
pertValue	50	Dual	52 (w/ PEsteepest), 56, 81
crash	off	Dual	idiot6 (degenerate)
idiot	-1	Primal	{7, 30, 60, 80, 100} (★)
sprint	-1	Primal	{1217, 3384} (cols $\gg$ rows)
primalPivot	automatic	Primal	PEsteepest (degenerate)
primalWeight	1e10	Primal	increase if many infeasibilities
cholesky	native	Barrier	UniversityOfFlorida (★)
gamma	0	Barrier	levels 1–4 (ill-cond.)
crossover	on	Barrier	off (no basis needed)
maxIterations	200/1000	Barrier	{500, 2000}
pertValue	50	Barrier	{51, 56, 61, 102, -208}

7 Empirical Study: Decision Trees for Algorithm Selection

Vilas Boas, Santos et al. [1] evaluated 532 CLP configurations on 1004 LP instances (all MIPLIB benchmarks, LP relaxations) and used integer programming to construct optimal decision trees for algorithm selection. Key results: 22% improvement with the single best configuration over CLP defaults; 40% improvement using the full depth-3 decision tree (cross-validated).

7.1 Most discriminating problem features

The features that appear at the root and high levels of the optimal decision tree are the *strongest predictors* of the best algorithm.

Feature	Description	Why it matters
aRatioLSA ★	Ratio of largest to smallest absolute non-zero in constraint matrix	Root split (most important). High ratio = ill-conditioned matrix; favours primal.
objMed	Median of objective coefficients	Near-zero median $\Rightarrow$ structured/near-degenerate objective; same as used by <code>guess()</code> .
rhsMax	Maximum RHS value	Large RHS $\Rightarrow$ large dual feasible region; favours primal crash.
objRatioLSA	Ratio of largest/smallest absolute objective coeff.	Measures objective conditioning; appears at level 3 (multiple nodes).
int	Number of integer variables	LP relaxation context: separates pure-continuous from mixed instances.

The `guess()` heuristic uses only `objMed/objAvg`, ignoring the most discriminating feature `aRatioLSA` (constraint matrix conditioning). For instance-specific tuning, compute $\text{aRatioLSA} = \max_{a_{ij} \neq 0} |a_{ij}| / \min_{a_{ij} \neq 0} |a_{ij}|$ and use it to guide algorithm selection.

7.2 The depth-3 decision tree

Figure 1 shows the structure of the paper’s depth-3 decision tree with 8 leaf nodes covering all 1004 instances. Each leaf shows the recommended algorithm configuration and number of instances assigned.

7.3 Configurations per leaf and parameters evaluated in Table 2

Instances	Method	Configuration
127	Primal	idiot=7
104	Dual	dualp=pesteeep
113	Dual	dualp=pesteeep, psi=1.0, scal=geo
77	Primal	idiot=80
149	Dual	crash=idiot6, dualp=steep, scal=rows, spars=off, subs=4354, pertv=81, passp=-81
173	Primal	idiot=100
139	Primal	idiot=100
122	Dual	dualp=pesteeep, psi=1.0, pertv=52

The paper explored the following ranges for each parameter (Table 2 of the paper):

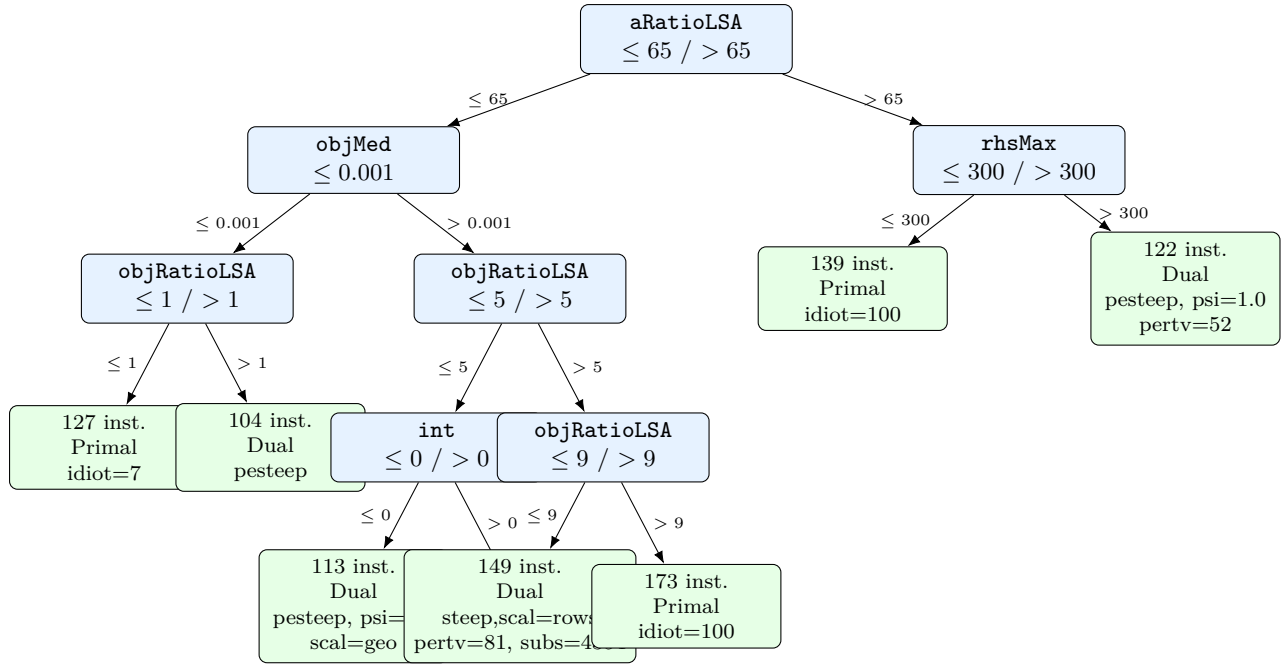


Figure 1: Depth-3 decision tree for CLP algorithm selection (Vilas Boas et al., ITOR 2019). 1004 LP relaxation instances; 40% improvement over defaults in cross-validation. Internal nodes show the splitting feature and threshold; leaves show recommended algorithm configuration and instance count.

Parameter	Values tested
<i>Primal simplex</i>	
idiot	{3,4,5,6,7,9,10,11}
pertv	{3500, -3157, -3000, -2395, -2000, -1483, -1000, 61}
sprint	{1217, 1557, 1804, 3384, 4826}
primalp	{change, exa(ct)}
psi	{-0.84, -0.62, -0.35, 0.62, 0.66, 0.84, 0.91}
scal	{geo, rows}
subs	{138, 294}
perturb	{1, 57, 15, 20, 25, 30, 35, 40, 50, 60, 80, 100}
passp	{22, 40, 80, 138, 294}
<i>Dual simplex</i>	
idiot/crash	{idiot1...7, on, lots, so}
dualize	{1}
pertv	{4900, ..., 820} (various including negatives)
dualp	{pesteepest, steep}
psi	{-1.1, ..., 1.1}
scal	{geo, rows}
passp	{167, -81, -67, -33, 0, 36, 67, 93}
subs	{37, 40, 41, 4354, 4392}
<i>Barrier</i>	
cholesky	{UniversityOfFlorida, dense}
pertv	{-208, -61, 50, 51, 56, 61, 102, 208}

8 Practical Tuning Recommendations

8.1 Quick decision guide

1. **Compute `aRatioLSA`** = max absolute coefficient / min absolute coefficient in the constraint matrix.
 - `aRatioLSA` $\leq$ 65: matrix is well-conditioned $\rightarrow$ follow the left branch of Figure 1 (check `objMed`).
 - `aRatioLSA` $>$ 65: matrix is ill-conditioned $\rightarrow$ right branch (check `rhsMax`).
2. **For dual simplex on degenerate problems:**

```
clp problem.lp -dualPivot PEsteepest -psi 0.1 -pertValue 52 \
               -scaling geo -duals
```

3. **For primal simplex on structured/integer-heavy LPs:**

```
clp problem.lp -idiot 60 -scaling geo -primal
```

4. **For barrier (large or difficult root LP), with AMD build:**

```
clp problem.lp -barrier -cholesky UniversityOfFlorida \
               -crossover on -scaling geo
```

5. **For long/thin problems (many columns, few rows):**

```
clp problem.lp -sprint 3384 -primal
```

8.2 Recommended parameter grid for instance-based tuning

The following grids can be used as a starting point for automated tuning (e.g. with `irace` or manual grid search):

Dual simplex

```
dualPivot : {automatic, PEsteepest, steep}
psi        : {0.1, 0.5, 1.0}                # 1.0 = PE off
pertValue  : {50, 52, 56, 81}
scaling    : {automatic, geo, rows}
passPresolve: {5, -33, -67, -81}
substitution: {3, 138, 4354}
crash      : {off, idiot6}
```

Primal simplex

```
idiot      : {0, 7, 30, 60, 80, 100}
pertValue  : {50, 61, -1483}
sprint     : {0, 1217, 3384}
scaling    : {automatic, geo, rows}
passPresolve: {5, 40, 80, 138}
```

Barrier

```
cholesky   : {native, UniversityOfFlorida}
pertValue  : {50, 51, 56, 61, 102, -208}
gamma      : {off, on (level 1), onstrong (level 4)}
crossover  : {on, off}
```

8.3 Notes for CBC branch-and-bound context

1. **Dual simplex at every B&B node:** warm-start from parent basis. Parameters affecting dual convergence and stability are most critical. `PEsteepest` + modest `pertValue` is the highest-leverage setting.
2. **dualBound** matters most at the root LP. Child-node LPs typically have tight bounds from branching, reducing the risk of large duals.
3. **Barrier for root LP only:** no warm-start between nodes. Use `-cholesky UniversityOfFlorida -crossover` on for large root LPs.
4. **Presolve effort at nodes:** reduce with `-passPresolve -33` or lower to avoid overhead at B&B nodes where gains are small.
5. **Idiot crash:** most beneficial for the first LP (root). Not useful at child nodes where a warm basis is available.

9 Default Values Reference

Parameter	Runtime default	Source location
<code>dualTolerance</code>	10^{-6}	<code>ClpModel.cpp:132</code>
<code>primalTolerance</code>	10^{-6}	<code>ClpModel.cpp:133</code>
<code>dualBound</code>	10^{10}	<code>ClpModel.cpp:61</code>
<code>perturbation_</code>	100 (auto on)	<code>ClpModel.cpp:113</code>
<code>maxFactor</code>	200 $\rightarrow$ auto-tuned	<code>CoinFactorization1.cpp:217</code>
auto-tune formula	$75 + \lfloor n/50 \rfloor$, cap 1000	<code>ClpSimplex.cpp:11315</code>
<code>dualRowPivot</code>	<code>ClpDualRowSteepestCliqueMode3</code>	<code>ClpModel.cpp:157</code>
<code>primalColPivot</code>	<code>ClpPrimalColumnSteepest(mode=3)</code>	
<code>scaling</code>	automatic	<code>ClpParameters.cpp</code>
<code>presolve</code>	on (5 passes)	<code>ClpParameters.cpp</code>
<code>substitution</code>	3	<code>ClpParameters.cpp:1400</code>
<code>idiot</code>	-1 (auto)	<code>ClpParameters.cpp</code>
<code>sprint</code>	-1 (auto)	<code>ClpParameters.cpp</code>
<code>psi (CLI)</code>	-0.5 (PE inactive)	<code>CbcOrClpParam.cpp:3592</code>
<code>psi (PEsteepest ctor)</code>	0.5	<code>ClpPEDualRowSteepest.hpp</code>
<code>psi (C API default)</code>	-1.0	<code>Cbc_C_Interface.cpp:4956</code>
<i>Barrier</i>		
<code>gamma_</code>	0.0	<code>ClpInterior.cpp:61</code>
<code>delta_</code>	0.0	<code>ClpInterior.cpp:62</code>
<code>maxBarrierIterations</code>	200 ($\rightarrow$ 1000 if large)	<code>ClpInterior.cpp:103</code>
<code>diagonalPerturbation</code>	10^{-15}	<code>ClpInterior.cpp:60</code>
<code>projectionTolerance</code>	10^{-7}	<code>ClpInterior.cpp</code>
<code>cholesky</code>	native	<code>ClpSolve.cpp:3140</code>

References

- [1] M. G. Vilas Boas, H. G. Santos, L. H. C. Merschmann, G. Vanden Berghe, *Optimal Decision Trees for the Algorithm Selection Problem: Integer Programming Based Approaches*, International Transactions in Operational Research, DOI: 10.1111/itor.12724, 2019.
- [2] M. Towhidi, J. Orban, F. Soumis, *The positive edge simplex method for linear programming*, INFORMS Journal on Computing 26(3):467–481, 2014.
- [3] COIN-OR Foundation, *COIN-OR Linear Programming (CLP)*, <https://github.com/coin-or/Clp>.
- [4] COIN-OR Foundation, *COIN-OR Branch and Cut (CBC)*, <https://github.com/coin-or/Cbc>.